



VIRTUE: Virtual Interactive Reality Toolkit for Understanding the EIC

User Guide V 3.1.1

Sean Preins
University of California, Riverside

Introduction

VIRTUE is a standalone customizable event display for collider experiments for desktop or virtual reality (VR). The application is available for free on GitHub, Steam, Google Play, and Zenodo, and features a simplified and engineering model of the ePIC detector for the future Electron-Ion Collider, as well as simulated electron-proton collisions. New collision data and detector geometries can be uploaded into the program as JSON or FBX files within the application folder.

<https://github.com/SeanBP/VIRTUE>

<https://store.steampowered.com/app/2728380/VIRTUE>

https://play.google.com/store/apps/details?id=com.Quantanaut.VIRTUE&hl=en_US

<https://doi.org/10.5281/zenodo.10372110>

System Requirements

The desktop version of the application is available for Windows, macOS, and Linux platforms. System requirements are determined by the Unity 6 runtime requirements:

<https://docs.unity3d.com/6000.2/Documentation/Manual/system-requirements.html>

The VR version of the application is compatible with Oculus Quest 2/3 headsets through Meta Horizon Link and with OpenXR-compatible VR systems, including SteamVR-supported headsets. The host Windows PC must satisfy the Unity 2022.3 system requirements:

<https://docs.unity3d.com/2022.3/Documentation/Manual/system-requirements.html>

When using Meta Horizon Link with a Quest headset, the additional Meta requirements are:

<https://www.meta.com/help/quest/140991407990979/>

For other OpenXR-compatible headsets, users should consult the requirements of the corresponding OpenXR runtime and VR platform.

The mobile version of the application is available for Android devices. System requirements are determined by the Unity 6 runtime requirements listed above.

Displaying Event Data

The program comes pre-loaded with a JSON file containing simulated electron-proton data in the ePIC detector at the EIC, as well as an example event data file. To display your own data, create a JSON file within the folder located at:

- **Windows:** "VIRTUE [Platform]_Data/StreamingAssets/Events/"
- **macOS:** Right-click on the application, select "Show Package Contents," and navigate to "Contents/Resources/Data/StreamingAssets/Events/"
- **Linux:** "VIRTUE [Platform]_Data/StreamingAssets/Events/"

The JSON files in this folder contain data for particle collision events within the virtual environment. For best performance, it is recommended to store only a small (< 100) number of events within each file, depending on how populated the events are.

Particle collision data can be generated directly from EDM4eic ROOT files using the Python conversion tools available in the VIRTUE GitHub repository:

<https://github.com/SeanBP/VIRTUE>

The conversion scripts read EDM4eic data products from CERN ROOT files and produce VIRTUE-compatible event JSON files containing tracks, hits, clusters, and other detector information.

The format of the JSON data file is as follows:

```

1  "header": { // Header contains metadata for the event data
2    "version": "3.1.1", // Version of the data format
3    "experiment": "Example Data", // Optional: Name of the experiment
4    "energy_unit": "GeV", // Optional: Unit for energy (default: GeV)
5    "color_bar": "Lin", // Optional: Type of color bar (Lin or Log)
6    "scale": 2.0, // Optional: Scales the size of the event
7    "length_unit": "mm", // Unit of length (e.g., m, cm, mm)
8    "particles": [ // Optional: Array of particle objects to be
        animated before the event
9      {
10        "size": 150.0, // Size of the particle
11        "color_rgba": [1.0, 0.0, 0.0, 1.0], // RGBA color values
12        "ip": [0.0, 0.0, 0.0], // Optional: Interaction point at t=0 (default: origin)
13        "angle_rad": [0.0, 3.14159] // Direction in radians
14      }
15    ],
16    "tracker_settings": {
17      "B_field_T": 2, // Optional: Magnetic field in Tesla (default: 0 T)
18      "segment_ns": 0.05, // Optional: Segmentation length of tracks. Increase
        this to improve performance.
19      "tracker_boundary": [10000.0, -20000.0, 20000.0]
20      // Cylindrical boundary for the tracker as an array of radius, left boundary,
        right boundary. All tracks will terminate outside of this boundary.
21    }
22  },
23  // Array of events containing event-specific data
24  "events": [
25    {
26      // Contains metadata for the event
27      "event_data": {
28        "info_text": "Example Event", // Optional: Event description
29        "energy_scale": [0, 10] // Optional: Energy range for the color bar
30      },
31      "hits": [ // Optional: Array of hit objects
32        {
33          "position": [1000, 0, 0], // Position of the hit
34          "time_ns": 10, // Optional: Time of the hit in nanoseconds
35          "size": 200.0, // Size of the hit
36          "color_rgba": [0.5, 0.0, 0.5, 0.8]
37          // Optional: RGBA color values for the hit
38        }
39      ],
40      "clusters": [ // Optional: Array of cluster objects
41        {
42          "position": [0, 1000, 1000], // Position of the cluster
43          "granularity": 300, // Granularity of the cluster
44          "length": 3000, // Length of the cluster
45          "time_ns": 20, // Optional: Time in nanoseconds
46          "color_rgba": [0.0, 0.0, 1.0, 1.0]
47          // Optional: RGBA color values for the cluster
48        }
49      ],
50    },
51  ],
52
53
54

```

```

55
56
57 "jets": [                                     // Optional: Array of jet objects
58   {
59     "length": 3000,                           // Length of the jet
60     "R_rad": 0.1,                             // Opening angle in radians
61     "angle_rad": [1.57, 1.57],                // Direction in radians
62     "vertex": [0, 1000, 0],                   // Optional: Vertex of the jet (default: origin)
63     "time_ns": 20,                            // Optional: Time in nanoseconds
64     "color_rgba": [0.0, 0.0, 1.0, 1.0]
65     // Optional: RGBA color values for the jet
66   }
67 ],
68 "tracks": [                                   // Optional: Array of track objects
69   {
70     "qOverP": -3,                             // Optional: Charge over momentum ratio (default
71       : 0)
72     "angle_rad": [2.3, -1.57],                // Direction of the track in radians
73     "vertex": [0.0, 500.0, 0.0],              // Optional: Starting position (default: origin)
74     "duration_ns": [5.0, 30.0],              // Optional: Start and end times
75     "color_rgba": [0.0, 0.0, 1.0, 1.0]
76     // Optional: RGBA color values for the track
77   }
78 ],
79 "blocks": [                                   // Optional: Array of block objects
80   {
81     "position": [1000, 1000, 1000],            // Position of the block
82     "time_ns": 20,                            // Optional: Time of the block in
83       nanoseconds
84     "size": [200, 400, 600],                  // Size along x, y, z axes
85     "euler_angles_deg": [100, 200, 300],
86     // Optional: Rotation in degrees along x, y, z axes
87     "color_rgba": [1, 0, 0, 1]               // Optional: RGBA color values for the block
88   }
89 ]

```

Displaying 3D Models

VIRTUE comes pre-loaded with a JSON file and FBX files containing the geometry of the ePIC detector at the EIC, as well as an example JSON 3D model file. To display a new 3D model, create a JSON or FBX file in the appropriate directory:

- **Windows:** VIRTUE [Platform]_Data/StreamingAssets/Models/
- **macOS:** Right-click on the application, select "Show Package Contents", and navigate to: Contents/Resources/Data/StreamingAssets/Models/
- **Linux:** VIRTUE [Platform]_Data/StreamingAssets/Models/

For Steam installations, locate the application folder by opening Steam, selecting VIRTUE in the library, clicking the gear icon, then selecting "Manage" → "Browse local files".

FBX Model Support

Many 3D computer-aided design (CAD) programs allow models to be exported in FBX (Filmbox) format. Alternatively, many CAD file formats can be converted to FBX using third-party software, including online file conversion tools. When exporting, ensure that textures, materials, and colors are embedded into the file.

FBX files in VIRTUE are loaded using the TriLib2 model loading package. To verify compatibility, upload your FBX file to the TriLib2 Model Viewer Web Demo: <https://ricardoreis.net/trilib/trilib2demo/modelviewer/>.

SketchUp FBX Export Procedure

The native FBX exporter in SketchUp often results in improperly rendered models. To ensure proper rendering, it is recommended to convert SketchUp files to FBX using the Unity Editor. The Unity Editor can be downloaded via the Unity Hub at <https://unity.com/download>. To correctly export a SketchUp model to FBX, follow these steps:

1. In SketchUp, **explode** all groups within the model and save.
2. In the Unity Editor, navigate to **Assets** → **Import New Asset...** and select the SketchUp (.skp) file.
3. In the **Assets** folder, create a new folder named **Color**.
4. Select the imported SketchUp file in the **Assets** folder. In the **Inspector** under the **Materials** tab, click "Extract Textures..." and "Extract Materials...", saving them in the **Color** folder.
5. Drag and drop the SketchUp asset into the Unity scene. Adjust its orientation and position as needed.
6. In the **Hierarchy**, right-click the model and select "Export To FBX...".
7. In the export options:
 - Set "Include" to "Model(s) Only".
 - Set "Object(s) Position" to "World Absolute".
 - Ensure "Embed Textures" is checked.
8. The exported FBX file is now ready for use in VIRTUE.

JSON-Based 3D Model Geometry

In addition to supporting FBX models, VIRTUE implements a custom JSON format for defining simplified detector geometries. This format enhances event visibility and allows for individual component labeling. It supports three component types: **toroid**, **block**, and **sphereoid** components.

For optimal visualization, components should be indexed from the innermost to the outermost structures. Circular components should be approximated with no more than 50 sides for improved performance. The expected JSON format follows this structure:

```
1  "header": { // Header contains metadata about the model configuration
2    "version": "3.1.1", // Version of the model format
3    "detector": "Example Model", // Optional: Name of the model
4    "length_unit": "m", // Unit of length (e.g., m, cm, mm)
5    "scale": 1.0 // Optional: Scale of the model
6  },
7  "components": [
8    {
9      "index": 0, // Index of the component to set the render order.
10     "type": "toroid", // Type of the component (toroid)
11     "name": "Example Toroid", // Optional: Name of the toroidal component
12     "position": [0, 0, 0], // Position in space (x, y, z coordinates)
13     "sides": 10, // Number of sides for the toroid
14     "radii": { // Radii for the left and right sides
15       "left": [0, 2], // Inner and outer radii for the left side
16       "right": [1, 2] // Inner and outer radii for the right side
17     },
18     "length": [1, 2], // Inner and outer length
19     "inner_offset": -1 // Optional: Offset of the inner barrel
20     "euler_angles_deg": [10, 20, 45],
21     // Optional: Rotation in degrees along x, y, z axes
22     "color_rgba": [1.0, 1.0, 0.5, 0.3], // RGBA color values
23   },
24   {
25     "index": 1,
26     "type": "block", // Type of the component (block)
27     "name": "Example Block", // Optional: Name of the block component
28     "position": [4, 0, 0], // Position in space (x, y, z coordinates)
29     "size": [1, 2, 3], // Size along x, y, z axes
30     "euler_angles_deg": [0, 45, 0],
31     // Optional: Rotation in degrees along x, y, z axes
32     "color_rgba": [0.5, 1.0, 1.0, 0.3] // RGBA color values
33   },
34   {
35     "index": 2,
36     "type": "sphereoid", // Type of the component (sphereoid)
37     "name": "Example Sphereoid", // Optional: Name of the sphereoid component
38     "position": [8, 0, 0], // Position in space (x, y, z coordinates)
39     "size": [1, 2, 3], // Size along x, y, z axes
40     "euler_angles_deg": [0, 45, 0],
41     // Optional: Rotation in degrees along x, y, z axes
42     "color_rgba": [1.0, 0.5, 1.0, 0.3] // RGBA color values
43   }
44 ]
```

Displaying Tours

The program comes pre-loaded with a JSON file containing "tour" information that will guide the user through different layers of the ePIC detector and collision events. To display your own tour, create a JSON file within the folder located at:

- **Windows:** "VIRTUE [Platform]_Data/StreamingAssets/Tours/"
- **macOS:** Right-click on the application, select "Show Package Contents," and navigate to "Contents/Resources/Data/StreamingAssets/Tours/"
- **Linux:** "VIRTUE [Platform]_Data/StreamingAssets/Tours/"

The JSON files in this folder contain data for scenes in a tour, which references a model and event file. The format of the JSON data file is as follows:

```
1  "header": {
2      "version": "3.1.1",                // Version of the data format
3      "model_file": "Example_Model.json", // Name of the model file
4      "events_file": "Example_Events.json", // Name of the events file
5  }
6  "scenes": [                            // Array of scenes in the tour
7      {
8          "model_settings":
9              "components": [0, 1, 2], // Indexes of the components to show
10             "lines_active": [0, 1], // Indexes of the components to show the
11                                     wireframe of
12         "text":{
13             "title": "Scene 1",                // Title of the scene
14             "body": "This shows a scene with no event." // Body of text to display
15                                     in the scene
16         },
17         "camera_settings":{
18             "position": [ 0.0, 2.0, -10.0 ], // Position of the camera
19             "focus": [ 0.0, 0.0, 0.0 ]      // Position of the focus of the camera
20         },
21         {
22             "model_settings": {
23                 "all_components": true // Toggle to display all components in the model
24             }
25             "text":{
26                 "title": "Scene 2",
27                 "body": "This shows a scene with an event."
28             },
29             "event_settings": {
30                 "index": 3,                // Index of the event to animate during the scene
31                 "time_before": 20,         // Nanoseconds to animate before the event
32                 "speed": 1                 // Speed of the animation
33             },
34             "camera_settings":{
35                 "position": [ 0.0, 2.0, -10.0 ],
36                 "focus": [ 0.0, 0.0, 0.0 ]
37             }
38         }
39 ]
```

Unity Projects

The Unity project files for both the desktop and VR versions of the application are available on Zenodo, to allow further customization if desired. <https://doi.org/10.5281/zenodo.10372110>

To edit the projects, download and unzip the project files and open them through the Unity Hub application: <https://unity.com/download>. The way the application renders and animates data from the JSON files can also be modified by editing the respective C# scripts in the assets folder of the project.

Once the desired edits are made to the project, build the project for your desired platform and run it from the executable file this generates. For further details, refer to the Unity documentation: <https://docs.unity.com/>.

Acknowledgments

Developer: Sean Preins

Supervisor: Prof. Kenneth Barish

<https://arratialab.ucr.edu/>

Electron-Ion Collider: www.bnl.gov/eic/

Simulations and detector design by the ePIC Collaboration:

www.bnl.gov/eic/epic.php

<https://eic.jlab.org/Menagerie/>

Music: Josh Puddington

Logo: Justyna Babinska "JustaSuta"

TriLib 2: Ricardo Reis

<https://ricardoreis.net/trilib-2/>

This work was supported by the University of California, Office of the President.